

[Dashboard](#) / [My courses](#) / [ITB_IF2010_2_2526](#) / [Tutorial 5](#) / [Interface, Generic, Generic wildcard, collection](#)

Started on	Tuesday, 5 May 2026, 6:50 PM
State	Finished
Completed on	Tuesday, 5 May 2026, 8:10 PM
Time taken	1 hour 19 mins
Grade	400.00 out of 400.00 (100%)

Question **1**

Correct

Mark 100.00 out of 100.00

Time limit	1 s
Memory limit	64 MB

Yu-Gi-Mon!

Para Nimon sedang bosan karena terlalu sering bermain Nimonspoli. Mereka kemudian meminta kalian untuk membantu mereka membuat sebuah game kartu yang jauh lebih seru bernama Yu-Gi-Mon! Di Game ini terdapat sebuah kartu [Monster](#). Kartu monster ini hanya merepresentasikan informasi kartu. Namun, kartu monster dapat di-*summon* ke *field*. Ketika berhasil di-*summon*, kartu bisa memiliki fungsi-fungsi tambahan. Agar kelas monster yang sudah ada tidak perlu diubah, bantulah Nimon mengimplementasikan interface `ISummoned` beserta kelas [SummonedMonster](#) untuk monster yang telah di-*summon*.

Spesifikasi File

Nama Kelas

Spesifikasi

Interface`ISummoned`

- Method:
 - `boolean flip()`:
Mengubah kondisi kartu dari tertutup menjadi terbuka kemudian mengembalikan `true`. Bila kartu sudah dalam keadaan terbuka, tidak terjadi apa-apa dan mengembalikan `false`
 - `void rotate()`:
Mengubah posisi kartu dari menyerang ke bertahan atau sebaliknya.
 - `int getPositionValue`:
Mengembalikan nilai `attack` jika posisi kartu menyerang. Mengembalikan nilai `defense` jika posisi kartu bertahan.
 - `void render()`:
Melakukan print ke terminal, sudah diimplementasi pada kelas `SummonedMonster` yang telah diberikan.
- Attributes:
 - `private Monster monster`:
Referensi ke informasi kartu monster.
 - `private boolean isFaceUp`:
True artinya kartu dalam keadaan terbuka. False artinya kartu dalam keadaan tertutup.
 - `private boolean isAttacking`:
True artinya kartu dalam posisi menyerang. False artinya kartu dalam posisi bertahan.
- Konstruktor: `public SummonedMonster(Monster m, boolean faceUp, boolean attackPos)`:
Inisialisasi semua Attributes.
- Method: Implementasi semua method dari interface `ISummoned`

Class`SummonedMonster`

Format Pengumpulan

Kumpulkan file: `ISummoned.java` dan `SummonedMonster.java` dalam 1 zip file.

Java 8 ▾

 [answer.zip](#)

Score: 100

Blackbox

Score: 100

Verdict: Accepted

Evaluator: Exact

No	Score	Verdict	Description
1	25	Accepted	0.06 sec, 28.73 MB

No	Score	Verdict	Description
2	25	Accepted	0.06 sec, 28.01 MB
3	25	Accepted	0.06 sec, 28.88 MB
4	25	Accepted	0.06 sec, 28.75 MB

Question **2**

Correct

Mark 100.00 out of 100.00

Time limit	1 s
Memory limit	64 MB

API

Dalam pengembangan *backend* modern, pertukaran data antara klien dan server selalu melibatkan ***Request*** (permintaan) dan ***Response*** (balasan). Tipe data yang dikirim dalam *request* sering kali berbeda dengan tipe data yang diterima dalam *response*. Misalnya, Anda mengirimkan *request* berupa kata kunci pencarian (***String***), lalu menerima *response* berupa ID hasil pencarian (***Integer***).

Tugas Anda:

1. Buat file ***APIResponse.java***

Berisi `public class APIResponse<U>` dengan spesifikasi:

- Memiliki atribut `private int statusCode`, `String message`, dan `U data`.
- Memiliki *constructor* untuk menginisialisasi ketiga atribut tersebut.
- Memiliki method `void printResponse()` yang mencetak dengan format:
`Response [statusCode] - [message] | Data: [data] (Type: [Nama Class dari U])`

2. Buat file ***APIRequest.java***

Berisi `public class APIRequest<T>` dengan spesifikasi:

- Memiliki atribut `private String endpoint` dan `T payload` (data yang dikirim).
- Memiliki *constructor* untuk menginisialisasi kedua atribut tersebut.
- Memiliki **Generic Method** bernama `execute`. Method ini menerima tipe generic baru `<U>` dengan parameter: `int statusCode`, `String message`, dan `U responseData`.
- Method `execute` ini harus melakukan dua hal:
 1. Mencetak log ke layar: `Executing Request to [endpoint] with payload: [payload]`
 2. Mengembalikan sebuah objek `APIResponse<U>` yang baru dibuat berdasarkan parameter yang diterimanya.

Driver

Gunakan kode di bawah ini tanpa mengubah isinya untuk melakukan pengetesan terhadap class yang telah Anda buat.

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        try {
            // Request(String) -> Response(Integer)
            String endpoint1 = scanner.nextLine();
            String payload1 = scanner.nextLine();
            int status1 = scanner.nextInt();
            scanner.nextLine();
            String message1 = scanner.nextLine();
            int resData1 = scanner.nextInt();
            scanner.nextLine();

            APIRequest<String> req1 = new APIRequest<>(endpoint1, payload1);
            APIResponse<Integer> res1 = req1.execute(status1, message1, resData1);
            res1.printResponse();

            // Request(Integer) -> Response(String)
            String endpoint2 = scanner.nextLine();
            int payload2 = scanner.nextInt();
            scanner.nextLine();
            int status2 = scanner.nextInt();
            scanner.nextLine();
            String message2 = scanner.nextLine();
            String resData2 = scanner.nextLine();

            APIRequest<Integer> req2 = new APIRequest<>(endpoint2, payload2);
            APIResponse<String> res2 = req2.execute(status2, message2, resData2);
            res2.printResponse();

        } catch (Exception e) {
            System.out.println("Kesalahan membaca input.");
        } finally {
            scanner.close();
        }
    }
}
```

Contoh Input:

```
/api/login
admin_user
200
Success
10543
/api/users/10543
10543
200
OK
admin_user
```

Contoh Output:

```
Executing Request to /api/login with payload: admin_user
Response 200 - Success | Data: 10543 (Type: Integer)
Executing Request to /api/users/10543 with payload: 10543
Response 200 - OK | Data: admin_user (Type: String)
```

Instruksi Pengumpulan:

Kumpulkan 2 file berikut yang disatukan ke dalam satu file **.zip**:

- **APIResponse.java**
- **APIRequest.java**

Java 8 ▾

 [answer.zip](#)

Score: 100

Blackbox

Score: 100

Verdict: Accepted

Evaluator: Exact

No	Score	Verdict	Description
1	10	Accepted	0.07 sec, 24.93 MB
2	10	Accepted	0.06 sec, 24.86 MB
3	10	Accepted	0.06 sec, 25.00 MB
4	10	Accepted	0.06 sec, 25.10 MB
5	10	Accepted	0.07 sec, 24.89 MB
6	10	Accepted	0.07 sec, 24.87 MB
7	10	Accepted	0.07 sec, 25.11 MB
8	10	Accepted	0.06 sec, 24.88 MB
9	10	Accepted	0.06 sec, 25.07 MB
10	10	Accepted	0.07 sec, 24.93 MB

Question **3**

Correct

Mark 100.00 out of 100.00

Time limit	1 s
Memory limit	64 MB

Nama File: KotakAjaib.zip

Para Nimon membutuhkan sistem penyimpanan barang yang fleksibel untuk gudang mereka. Gudang ini menyimpan berbagai jenis barang: **Elektronik**, **Makanan**, dan **Pakaian**. Setiap jenis barang disimpan di kotak yang berbeda, tetapi terdapat satu *kotak campuran* yang dapat menampung semua jenis barang.

Untuk mengelola kotak-kotak ini secara efisien, dibutuhkan *method utilitas* yang dapat bekerja dengan kotak bertipe apapun. Di sinilah konsep **Java Wildcard** berperan:

- `? extends T` (upper-bounded): untuk *membaca* dari kotak bertipe apa saja yang merupakan subtype dari T
- `? super T` (lower-bounded): untuk *menulis* ke kotak bertipe apa saja yang merupakan supertype dari T
- `?` (unbounded): untuk operasi yang tidak bergantung pada tipe spesifik

Kamu diminta mengimplementasikan file-file [berikut](#). **Main.java** sudah disediakan dan **tidak boleh diubah**. Kumpulkan kembali KotakAjaib.zip yang berisi semua file .java yang kamu buat, termasuk Main.java.

Spesifikasi

1. Hierarki Kelas Barang

Buat hierarki kelas dengan **Barang** sebagai abstract base class:

- **Barang** (abstract)
Atribut: `String nama, int harga`
Constructor: `Barang(String nama, int harga)`
Getter: `getNama(), getHarga()`
Abstract method: `String info()`
- **Elektronik** extends Barang
Atribut tambahan: `int watt`
Constructor: `Elektronik(String nama, int harga, int watt)`
Getter: `getWatt()`
`info()` → "[Elektronik] nama - harga IDR (wattW)"
- **Makanan** extends Barang
Atribut tambahan: `int kalori`
Constructor: `Makanan(String nama, int harga, int kalori)`
Getter: `getKalori()`
`info()` → "[Makanan] nama - harga IDR (kalori kal)"
- **Pakaian** extends Barang
Atribut tambahan: `String ukuran`
Constructor: `Pakaian(String nama, int harga, String ukuran)`
Getter: `getUkuran()`
`info()` → "[Pakaian] nama - harga IDR (ukuran)"

2. Generic Class Kotak<T extends Barang>

Kelas generik `Kotak<T extends Barang>` menyimpan item menggunakan `ArrayList<T>` dengan kapasitas terbatas. Implementasikan method berikut:

- `Kotak(int kapasitas)`: Konstruktork dengan kapasitas maksimum
- `boolean tambah(T item)`: Menambahkan item. Mengembalikan `true` jika berhasil, `false` jika penuh
- `T ambil()`: Mengambil dan menghapus item terakhir (LIFO). Mengembalikan `null` jika kosong
- `T lihat(int index)`: Melihat item pada index tertentu tanpa menghapus. Mengembalikan `null` jika index tidak valid
- `int jumlah()`: Mengembalikan jumlah item saat ini
- `int kapasitas()`: Mengembalikan kapasitas maksimum
- `boolean penuh()`: Mengembalikan `true` jika jumlah item sama dengan kapasitas
- `boolean kosong()`: Mengembalikan `true` jika kotak kosong

3. Kelas KotakUtils (Wildcard)

Kelas utilitas dengan method **static** yang menggunakan *wildcard*. Perhatikan tipe wildcard pada setiap method signature:

- `static void tampilkanSemua(Kotak<? extends Barang> kotak)`
Mencetak `info()` setiap item (satu per baris). Jika kosong, cetak "**Kotak kosong**".
Menggunakan upper-bounded wildcard karena hanya membaca dari kotak.
- `static int totalHarga(Kotak<? extends Barang> kotak)`
Mengembalikan total harga semua item dalam kotak.
Menggunakan upper-bounded wildcard karena hanya membaca harga.
- `static Barang termahal(Kotak<? extends Barang> kotak)`

Mengembalikan item dengan harga tertinggi, atau `null` jika kosong.
Menggunakan upper-bounded wildcard karena membaca dan membandingkan item.

- `static <T extends Barang> int pindahkan(Kotak<? extends T> src, Kotak<? super T> dst)`
Memindahkan semua item dari `src` ke `dst` secara LIFO. Berhenti jika `src` kosong atau `dst` penuh. Mengembalikan jumlah item yang dipindahkan.
Menggabungkan upper-bounded dan lower-bounded wildcard (prinsip PECS: Producer Extends, Consumer Super).
- `static int hitungItem(Kotak<?> kotak)`
Mengembalikan jumlah item dalam kotak.
Menggunakan unbounded wildcard karena tidak perlu tahu tipe spesifik.

Format Masukan

Baris pertama berisi empat integer `capE capM capP capC`: kapasitas kotak Elektronik, Makanan, Pakaian, dan Campuran (bertipe `Kotak<Barang>`). Baris kedua berisi integer `Q`. Selanjutnya `Q` baris perintah:

- `ADDE nama harga watt`: tambah Elektronik ke kotak 1
- `ADDM nama harga kalori`: tambah Makanan ke kotak 2
- `ADDP nama harga ukuran`: tambah Pakaian ke kotak 3
- `PRINT n`: cetak isi kotak ke-`n` (1=Elektronik, 2=Makanan, 3=Pakaian, 4=Campuran)
- `TOTAL n`: cetak total harga kotak ke-`n`
- `MAHAL n`: cetak item termahal di kotak ke-`n`
- `COUNT n`: cetak jumlah item kotak ke-`n`
- `TRANSFER n`: pindahkan semua item dari kotak ke-`n` ke kotak Campuran

Format Keluaran

Perintah	Kondisi	Keluaran
ADDE / ADDM / ADDP	berhasil	Ditambahkan: <code>info()</code>
ADDE / ADDM / ADDP	kotak penuh	Gagal: <code>Kotak penuh!</code>
PRINT n	ada isi	<code>info()</code> setiap item, satu per baris
PRINT n	kosong	<code>Kotak kosong</code>
TOTAL n	-	<code>Total: X</code>
MAHAL n	ada isi	<code>info()</code> item termahal
MAHAL n	kosong	<code>Kotak kosong</code>
COUNT n	-	<code>Jumlah: X</code>
TRANSFER n	-	<code>Dipindahkan: X item</code>

Contoh Masukan dan Keluaran

Masukan 1:

```
3 3 3 10
12
ADDE Laptop 15000 65
ADDE Charger 500 10
ADDM Roti 20 250
ADDM Susu 15 150
ADDP Kaos 100 M
ADDP Jaket 300 L
PRINT 1
PRINT 2
PRINT 3
TOTAL 1
TOTAL 2
COUNT 3
```

Keluaran 1:

```
Ditambahkan: [Elektronik] Laptop - 15000 IDR (65W)
Ditambahkan: [Elektronik] Charger - 500 IDR (10W)
Ditambahkan: [Makanan] Roti - 20 IDR (250 kal)
Ditambahkan: [Makanan] Susu - 15 IDR (150 kal)
Ditambahkan: [Pakaian] Kaos - 100 IDR (M)
Ditambahkan: [Pakaian] Jaket - 300 IDR (L)
```



```
[Elektronik] Laptop - 15000 IDR (65W)
[Elektronik] Charger - 500 IDR (10W)
[Makanan] Roti - 20 IDR (250 kal)
[Makanan] Susu - 15 IDR (150 kal)
[Pakaian] Kaos - 100 IDR (M)
[Pakaian] Jaket - 300 IDR (L)
Total: 15500
Total: 35
Jumlah: 2
```

Masukan 2:

```
3 3 3 5
10
ADDE Laptop 15000 65
ADDE Mouse 200 5
ADDM Roti 20 250
ADDP Kemeja 250 L
TRANSFER 1
PRINT 1
PRINT 4
TRANSFER 2
TRANSFER 3
PRINT 4
```

Keluaran 2:

```
Ditambahkan: [Elektronik] Laptop - 15000 IDR (65W)
Ditambahkan: [Elektronik] Mouse - 200 IDR (5W)
Ditambahkan: [Makanan] Roti - 20 IDR (250 kal)
Ditambahkan: [Pakaian] Kemeja - 250 IDR (L)
Dipindahkan: 2 item
Kotak kosong
[Elektronik] Mouse - 200 IDR (5W)
[Elektronik] Laptop - 15000 IDR (65W)
Dipindahkan: 1 item
Dipindahkan: 1 item
[Elektronik] Mouse - 200 IDR (5W)
[Elektronik] Laptop - 15000 IDR (65W)
[Makanan] Roti - 20 IDR (250 kal)
[Pakaian] Kemeja - 250 IDR (L)
```

Java 8

 [KotakAjaib.zip](#)

Score: 100

Blackbox

Score: 100

Verdict: Accepted

Evaluator: Exact

No	Score	Verdict	Description
1	20	Accepted	0.14 sec, 25.12 MB
2	20	Accepted	0.13 sec, 25.06 MB
3	20	Accepted	0.09 sec, 25.10 MB
4	20	Accepted	0.11 sec, 25.24 MB
5	20	Accepted	0.11 sec, 25.00 MB

Question **4**

Correct

Mark 100.00 out of 100.00

Time limit	1 s
Memory limit	64 MB

Anda diminta untuk melengkapi implementasi program untuk manajemen perpustakaan. Implementasi program ini akan memanfaatkan pemahaman anda mengenai Collection API (Interface, Iterator, Algorithm) dan juga Stream API. Kumpulkan file SmartLibrary.java dan LibraryIterator.java pada solution.zip. Berikut adalah [file](#) yang anda butuhkan

Java 8

 [solution.zip](#)

Score: 100

Blackbox

Score: 100

Verdict: Accepted

Evaluator: Exact

No	Score	Verdict	Description
1	10	Accepted	0.40 sec, 29.64 MB
2	10	Accepted	0.34 sec, 27.90 MB
3	10	Accepted	0.44 sec, 32.28 MB
4	10	Accepted	0.20 sec, 27.99 MB
5	10	Accepted	0.30 sec, 33.55 MB
6	10	Accepted	0.29 sec, 34.82 MB
7	10	Accepted	0.20 sec, 34.06 MB
8	10	Accepted	0.23 sec, 34.20 MB
9	10	Accepted	0.18 sec, 32.29 MB
10	10	Accepted	0.81 sec, 34.05 MB

◀ Slide Tutorial 5

Jump to...